Ders 4 Genişletilmiş Notu: Risk Değerlendirme, Gantt Diyagramı ve Faaliyet İlişkileri

Ders 4 Genişletilmiş Notu: Risk Değerlendirme, Gantt Diyagramı ve Faaliyet İlişkileri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Derslerin Özeti: Etkin Proje Yönetimi ve 3P
- 2. Proje Planının Genel Yapısı
- 3. Sorunun Tanımlanması
- 4. Çözüm Yolunun Kararlaştırılması
- 5. Geliştirme Sürecinin Planlanması
- 6. Proje Kapsamı ve Planın Sonraki Adımları
- 7. İş Kırılım Yapısı (WBS)
- 8. Zaman Çizelgesi, İş Gücü, Bütçe ve Kalite Planı
- 9. Yazılım Riski Nedir?
- 10. Risk Kategorileri
- 11. Risk Derecesi: Olasılık ve Etki
- 12. Riski Azaltma Örnekleri
- 13. Risk Tablosu Örneği ve Sadeleştirme
- 14. Gantt Diyagramı
- 15. Örnek: Kitap Satışı Web Sitesi
- 16. WBS Kaç Düzey Olmalı?
- 17. Faaliyetler Arası İlişkiler ve Kritik Yol
- 18. Proje Planlama Araçları

1. Önceki Derslerin Özeti: Etkin Proje Yönetimi ve 3P

Hoca derse, ilk haftalardan bu yana görülenleri özetleyerek başladı. Yazılım proje yönetimi birkaç bilim dalının arasında yer alır. Bu yüzden yazılım proje yöneticisinin de bu dallar çerçevesinde bilgi sahibi olması gerekir.

Etkin proje yönetimi için üç bileşen önemlidir. Bunlara 3P denir: insan kaynakları (people), problem (problem) ve süreç (process).

- **İnsan kaynakları:** İnsanların iyi seçilmesi, performansın iyi yönetilmesi, eğitimlerin ve kariyer geliştirme eğitimlerinin iyi takip edilmesi, organizasyonun ve iş tasarımının iyi kurulması, ekip kültürünün oluşturulması bu başlığın altındadır.
- **Problem:** Bir projeyi iyi yönetmek için çözülecek problemi iyi anlamak gerekir. Projenin amacı, alternatif çözümler, teknik kısıtlar ve yönetim kısıtları bu başlığın sorularıdır.
- **Süreç:** Yazılımın hangi süreçle geliştirileceği sorusudur. Hoca bu başlığı bu özette ayrıca açmadı, proje planı anlatımında geliştirme süreci olarak ele aldı.

Hoca burada problem başlığını risk değerlendirmeye bağladı. Kısıtlar ve varsayımlar üzerinden gidildiğinde, projedeki teknik ve yönetim kısıtları da riskler gibi sınıflanır ve risk olarak değerlendirilir.

> **Ek bilgi (derste anlatılmadı):** Pressman'ın kitabında aynı çerçeve büyük olasılıkla dört başlıkla verilir: insanlar (people), ürün (product), süreç (process) ve proje (project). Hocanın 3P çerçevesindeki problem başlığı, büyük olasılıkla kitaptaki ürün başlığının kapsamıyla örtüşür.

Genel proje yönetimi yol haritasında (Project Management Roadmap) ilk iki adım şudur: Önce projenin neden yapıldığı ve amacı anlaşılır, yani proje tanımlanır. Sonra projeyle ilişkili ayrıntılar tanımlanır. Proje planı ancak bundan sonra yapılabilir.

2. Proje Planının Genel Yapısı

Proje planı, göreve başlandığında kaynakların neler olduğunu, işin nasıl ve hangi sürede geliştirileceğini anlatır. Bu zaman ve kaynak planı, kurallara göre seçilen diyagramlarla gösterilir. Plan, yazılım yöneticisinin bilgi ve deneyimine dayanan tahminleri gerektirir. Daha önce ölçümler yapılmışsa değerlendirmeler bunlara göre yapılır. İşe başlandıktan sonra plan yeniden gözden geçirilip değiştirilebilir.

Hoca, IEEE proje yönetim planı düzeyinde çok ayrıntılı bir yapı yerine daha kısa bir yapı kullandı. Bu kısa yapının bölümleri şunlardır:

- **Konu:** Projenin amacı, hedefleri, ana işlevleri ve özellikleri. Geliştirme süreci olarak neyin kullanılacağı ve neden kullanılacağı da burada yazılır. Hangi süreç modellerinin kullanılabileceği önceki derste hatırlatılmıştı.
- **Kaynaklar:** İnsanlar (kullanıcılar, müşteriler, yazılım geliştiriciler), bilgisayar donanımı, yazılım ve zaman.
- **Risk yönetimi:** Bu projedeki riskler. Konu anlatılırken de projenin kısıtları ve varsayımları belirtilir. Kısıtlar ve varsayımlar da risk tanımlarına benzer biçimde tanımlanabilir.
- **Gider:** Bütçe, kaynaklara bağlı olarak hesaplanır. Ne kadar zaman, ne kadar ve hangi tip insan, hangi yazılım ve donanım gerektiği belli olunca gider ortaya çıkar.
- **Çizelgeler:** Bunlar çizelgelere dökülürken ağırlıklı olarak PERT ve Gantt diyagramları kullanılır. Hoca bu derste bunlara başlangıç yapacağını, sonra yeniden döneceğini söyledi.

Özetle, yazılım geliştirme planı üç soruyu yanıtlamalıdır: **ne**, **ne kadar** ve **ne zaman**.

Yazılım proje geliştirme planı şu adımları izler: Önce sorun tanımlanır, yani problemin ne olduğu ve hangi çözüm yoluyla çözüleceği belirlenir. Bir problemin birçok alternatif çözümü olabilir. Bunlardan en uygunu kararlaştırıldıktan sonra geliştirme süreci planlanır. Hangi model uygulanacaksa adımlar ona göre planlanır. Ayrıca proje organizasyonu ve proje zamanlaması da yapılır.

3. Sorunun Tanımlanması

Yazılım geliştirme projesinin ilk aşamalarında sorun, müşterinin istekleri yönünden belirlenir. Müşterinin bir hedefi vardır: maaş bordrosunun hazırlanması, envanterin hazırlanması ya da trafik kontrolünün gerçekleştirilmesi gibi. Beklentilerin neler olduğu, işin içine nelerin gireceği ve çözüme hangi sırayla geçileceği belirlenir. Kullanılacak algoritmalar, veri tabanı ilişkileri ve müşteriye verilecek bilgi ve belgeler de bu aşamada ele alınır. Ardından plan müşteriyle birlikte tekrar gözden geçirilir.

Burada problem genel olarak anlatıldığı için çok fazla teknik ayrıntıya girilmemesi öğütlenir. Algoritmalardan söz edilse de sorun, müşterinin anlayacağı biçimde ve özlü olarak tanımlanmalıdır.

3.1 Koşullar, Kısıtlar ve Varsayımlar

Sorun tanımlanırken koşullar ve kısıtlar da belirlenir. Bunlar projeyle ilgili kısıtlamalar ya da kabullerdir. Hoca üç örnek verdi:

- **Varsayım örneği:** Proje süresince kimsenin rapor almayacağı düşünülerek çalışılır. Bir problem olursa plana eklenir. Bu, proje kaynaklarına getirilen bir varsayımdır.
- **Kısıt örneği:** Bu projede herkesin mesai saatleri içinde çalışması istenir. Fazla mesai öngörülmez. Proje, fazla mesainin gündeme gelmeyeceği bir kısıtla başlar.

- **Kaynak kısıtı örneği:** Projede network sorunları için uzman gerekir. Ama uzman yalnızca belirli tarihlerde ve belirli saatlerde destek verir.

Bu örnekler projeyle ilişkili kısıtlardır. Bunların dışında bir de teknik ya da problemle ilişkili kısıtlar vardır. Bunlar geliştirilecek yazılımın konusu olan olayla ilişkili kısıtlar ve varsayımlardır.

3.2 Gereksinimler

Buradaki gereksinimler iki grubu kapsar. Birincisi, yazılım mühendisliğinden bilinen gereksinim analizi çerçevesindeki gereksinimlerdir. Problemi anlatan işlevlerin neler olduğu, hangi işlemlerin geliştirileceği gereksinim analizinin sonucundan öğrenilir. İkincisi, projede gerekli olan araç gereçtir: standart gereksinimleri, yazılım ve donanım kaynak gereksinimleri.

3.3 Kilometre Taşları

Sorunun tanımlanması aşamasında, amaçlara ulaşma ve gereksinimleri yerine getirme mekanizması da belirlenmelidir. Yani yazılım geliştirme sürecinin hangi dönüm noktalarında gözden geçirme yapılacağı ve kilometre taşlarının (milestone) neler olduğu söylenir.

Geliştirmenin adımları gereksinim analizi, tasarım, sınama ve bakımdır. Bu işlemlerin her birinin bitiş zamanı birer kilometre taşı olarak belirlenebilir. Her kilometre taşında bir gözden geçirme (review) yapılacağı söylenebilir. Bu kilometre taşları proje planına da yerleştirilir.

3.4 Çözüm Olanakları ve Kabul Kriterleri

Sorun müşteriyle birlikte tanımlandıktan sonra, yazılım geliştiriciler bilgisayar desteğiyle çözüm olanağını belirler. Donanım, yazılım ve kullanıcı öğelerinin işlevleri, kısıtları ve aralarındaki uyum belirtilir. Yazılımın geliştirilmesi ve üretilmesindeki hedefler, çok ayrıntıya inmeden yukarıdan bakılarak belirlenir. Gereksinimler saptanır ve kabul kriterleri belirlenir.

Kabul kriteri şu soruların cevabıdır: Müşterinin ilk baştaki tanımlamalarına göre hangi özelliklerde bir yazılım geliştirilecek? Yazılım bittiğinde hangi olanaklara sahip olacak? Müşteri neleri kontrol ederek teslim alacak?

> **Hocanın vurgusu:** Kabul kriterleri oldukça ayrıntılı anlatılmalı ve kayda geçirilmelidir. Yoksa iki taraftan biri memnun kalmayabilir ve iş bilirkişilik olayına dönüşebilir.

Bu adımlardan sonra elde şunlar vardır: Gereksinimler belirlenmiştir. Yazılım mühendisi gereksinimlerden yola çıkarak çözüm olanaklarını çıkarmıştır. Hedefler buna göre gözden geçirilmiştir. Sorun anlaşılmıştır. Sıradaki soru çözümün nasıl olacağıdır.

4. Çözüm Yolunun Kararlaştırılması

Çözüm yolunda önce olası çözümler, yani alternatifler belirlenir. Bunlardan biri seçilecektir. Daha fizibilite analizi sırasında projenin ana konusu olan probleme hangi çözüm alternatiflerinin getirilebileceği sorulur. Bu alternatiflerin hepsi için bir yapılabilirlik analizi oluşturulur. Alternatifler değerlendirilip karşılaştırılır. En uygun çözüm yolu, farklı yaklaşımların karşılaştırılmasıyla ortaya çıkar.

Bu karar verilirken ekip hâlinde çalışmak ve birlikte tartışmak önerilir. Önerilen her çözüm yolunun yapılabilirliği hem teknik hem de ekonomik olarak değerlendirilmelidir. Hoca fizibilite çalışmalarını, proje yönetim yol haritası tamamlandıktan sonra daha ayrıntılı anlatacağını söyledi.

Bir çözüm yolu bulmak için üç şeye bakılır: Hangi kaynaklar sağlanıyor, kurumda hangi teknoloji düzeyi kullanılıyor, proje hangi süre içinde isteniyor? Yazılımın hangi yolla geliştirileceği bunlara göre belirtilir. Fizibilite çalışması çok erken evrelerde de yapılabilir. Ama biraz veri toplayarak yapmak işin gerçekçiliğini artırır.

Çözüm yolu kararlaştırılırken şu yollardan biri kullanılır:

- Durum incelemesi.
- En kötü durum analizi.

- Simülasyon.
- Model oluşturma.

Seçeneklerden biri kabul edildiğinde, bunun neden kabul edildiği ve diğerlerinin neden reddedildiği de açıklanmalıdır.

> **Hocanın vurgusu:** Seçilen ve reddedilen çözümlerin gerekçesi hem bu proje hem de sonraki projelerin bilgi arşivi için gereklidir. Böylece yapılan çalışma tek yerde kalmaz, başka olaylara da çözüm götürür.

Önerilen çözüm yolunda üretimin öncelik sırası da liste hâlinde belirtilmelidir. Yazılım geliştirme sürecinde gereksinimlerde uyumsuzluklar olabilir. Bunları ortadan kaldırmak için önce listelemek gerekir. Hem yazılım geliştirmede hem de ürünlerde öncelik sırası oluşturulmazsa yanlış kararlar alınabilir. Hangi işlerin yapılacağı ve hangi yazılımların geliştirileceği, şirket stratejisi düşünülerek sıralanmalıdır.

5. Geliştirme Sürecinin Planlanması

Planlama aşamasında önce geliştirme süreci modeli kurulur. Sonra şunlar planlanır:

- Konfigürasyon yönetiminin ve kalite kontrolünün nasıl yapılacağı.
- Kullanılacak yazılım araçları ve yöntemler.
- Hangi gösterim şekillerinin kullanılacağı ve hangi grafiklerin çizileceği.
- Gider tahminleri.
- Zamanlama ve programın düzenlenmesi.
- Sistemin işletilmesi ve bakımı için gereken kaynakların nasıl tahmin edileceği ve hangi kaynakların kullanılacağı.

Hoca yazılım ölçümünün sonraki haftaların konusu olduğunu söyledi.

6. Proje Kapsamı ve Planın Sonraki Adımları

Proje planının içinde bir de risk analizi vardır. Hoca bunu göstermeden önce projenin kapsamını yeniden özetledi.

Projenin kapsamında hedefler, sınırlar, kısıtlar ve varsayımlar belirtilir. Hangi yazılım geliştirme süreç modelinin kullanılacağı yazılır. İlk başta proje bir kez daha açıklanabilir. Bu projede hangi yazılımların geliştirileceği ve kullanıma alınacağı olabildiğince vurgulanmalıdır. Varsayımlar ve kısıtlar bölümünde projenin çalışabilmesi için gerekli kısıtlardan söz edilir.

Planın akışı şöyledir: Süreç modeli seçilir, konu anlaşılır, projenin hangi konuda yapıldığı ve teslim edilecekler söylenir. Sonra risk analizi gerçekleştirilir. Risk analizinden sonra projenin alt aktiviteleri belirlenir. Ardından zaman çizelgesi, aktiviteler arası ilişkiler, iş gücü belirleme ve görev atama gelir.

6.1 Temel Aktiviteler ve Analizin Alt Adımları

Yazılım geliştirirken temel aktiviteler sırasıyla analiz, tasarım, gerçekleştirme ve testtir. Ardından bakım gelir. Bu aktivitelerin de alt aktiviteleri vardır. Örneğin sistem analizi ya da gereksinim analizi aşamasının alt başlıkları nelerdir? Hangi işler yapılıyorsa alt başlıkları da yazılmalıdır.

Analizin alt başlıkları Pressman'ın *Software Engineering* kitabında daha ayrıntılıdır. Hoca bunu genelleştirerek üç temel adıma indirdi:

1. Bilgi toplama.
2. Modelleme.
3. Bu modellerin yorumlanması.

> **Hocanın vurgusu:** Esas olan ayrıntılı listedir. Ama bu üç adım akılda tutulursa ayrıntılı adımları saymak da kolaylaşır.

7. İş Kırılım Yapısı (WBS)

İş kırılım yapısı (Work Breakdown Structure, WBS), projedeki iş adımlarını seviyelendirerek gösterir. Hoca bunu önceki hafta da anlatmıştı.

Hocanın örneği şuydu: Size "Üçüncü düzeye kadar ayrıntıyla spiral modelin WBS yapısını yazınız" dense ne yapılır? Önce spiral modele göre yazılım geliştirmede hangi alt başlıkların olduğu anlaşılır. Sonra bunlara göre bir Gantt diyagramı çizilir.

WBS iki biçimde gösterilebilir. Maddeler hâlinde, seviyeleri numaralandırarak yazılabilir. Ya da blok diyagramlarla, işler hiyerarşik olarak çizilebilir. İnceleyen açısından ikisi arasında fark yoktur.

> **Hocanın vurgusu:** Önemli olan, iş kırılım yapısının hiyerarşik gösterilmesi ve bu hiyerarşinin yapılan işle anlamlı olmasıdır. Alt işlemlerin seçilebileceği bir yapı kurulmalıdır.

8. Zaman Çizelgesi, İş Gücü, Bütçe ve Kalite Planı

Zaman çizelgesi oluşturulurken WBS yapısındaki her kırılım seviyesinin karşısına şunlar yazılır: Ne zaman başlayacağı, hangi alt işlemlerle ilişki içinde olacağı ve bu bağlantıların kuralı. Böylece zaman çizelgesi, Gantt diyagramı olarak karşımıza çıkar.

Aktiviteler arası ilişkilerin birkaç kuralı vardır. Biri bitmeden diğeri başlamamalıdır. İkisi aynı anda başlayabilir. Bir iş başladıktan belli bir süre sonra diğeri başlayabilir. WBS iyi oluşturulduktan sonra, yanına şu soruların cevabı da yazılmalıdır: Hangi alt işlem hangi zamanda başlayabilir? Hangi işin bitmesini bekleyecek? Hangi işle paralel olmalı?

İş gücü belirlemede yine WBS kullanılır. Her alt işlem için ne kadar iş gücü gerektiği söylenir. Benzer projeleri daha önce geliştirmiş kişilere tahmin ettirmek bir yoldur. Ardından görev atama ve bütçe gelir.

Projenin kalite planı da belirtilmelidir. Kalite planı, hangi gözden geçirmelerin hangi takvim çerçevesinde yapılacağını ve gözden geçirmeye kimlerin katılacağını söyleyen kurallardır. Bu planı yazılım geliştiren ekip oluşturur. Bu çalışmaların sonunda yazılım gereksinim spesifikasyonu (Software Requirements Specification) ortaya çıkar.

9. Yazılım Riski Nedir?

Risk şu anda bir problem değildir, ama gelecekte problem olabilir. İşlerin şimdi kötüye gitmesine neden olan bir özellik değildir. Gelecekte kötüye gitmesine neden olabilir. Hoca riski üç kelimeyle açıkladı: **gelecek**, **değişim** ve **seçim**.

Şu anda problem yoksa neden gelecekte olsun? Çünkü değişim vardır. Müşteri gereksinimleri değişebilir. Geliştirme teknolojileri değişebilir. Projeye bağlı varlıklar değişebilir. Bunlar değiştiğinde riskler de beraberinde gelebilir.

Proje içinde yaptığımız seçimler de bizi riske götürebilir. Hangi yöntem ve araçların kullanılacağı, kaç insanın ve kimlerin çalışacağı, kalite ölçütlerinden hangilerine önem verileceği birer seçimdir.

Hoca kalite ölçütlerinden birkaçını saydı. Biri yazılımın ne kadar bağımsız olduğudur. Bir diğeri yeniden kullanılabilirliktir (reusability). Yazılım bu özellikle geliştiriliyorsa, başka projelerde daha önce geliştirilip kütüphaneye yerleştirilmiş alt ürünler kullanılacak mı? Bu belirtilmelidir.

Yazılım risklerinin iki genel özelliği vardır: **belirsizlik** ve **kayıp**. İkisi de dikkate alınmalıdır.

10. Risk Kategorileri

Riskler genel olarak üç grupta özetlenebilir: proje riskleri, teknik riskler ve işletmecilik riskleri. Hoca bunun yanında, Pressman'ın kitabının önceki baskılarındaki daha ayrıntılı sınıflamayı kullandı. Bu alt sınıflar şunlardır:

- Ürün büyüklüğü riskleri.

- İşletmecilik riskleri.
- Müşteriyle ilişkili riskler.
- Süreç riskleri.
- Teknoloji riskleri.
- Geliştirme ortamı riskleri.
- Personel riskleri.

Bunların birçoğu proje, teknik ve işletmecilik üst başlıklarının altına da girebilir. Hoca ardından ürün büyüklüğü, işletmecilik, müşteri, süreç, teknik ve personel risklerini sırayla açıkladı. Çoğu, bir soru listesiyle belirlenir.

10.1 Ürün Büyüklüğü Riski

Yazılımın tahmin edilen büyüklüğü, satır sayısı ya da işlev puanı (function point) olarak ne durumdadır? Program sayısı ve dosya sayısı nedir? Hoca yazılım ölçümü tekniklerinin üzerinde daha sonra duracağını söyledi.

Ölçüt, ekibin önceki deneyimidir. Bu büyüklük ekibin daha önce geliştirdiği ürünlerin ortalamasından büyükse, ortalamadan sapma yüzdesi önem kazanır ve projede risk olabilir. Çünkü ekip alışmadığı büyüklükte bir yazılım geliştirmektedir.

Aynı şey başka büyüklükler için de geçerlidir:

- **Veri tabanı büyüklüğü:** Önceki ürünlerin kullandığı veri tabanları bu kadar büyük değilse, şimdi olmasa bile daha sonra problem çıkabilir.
- **Kullanıcı sayısı:** Ürünü kaç kişinin kullanacağı ve aynı anda kaç kişinin kullanacağı önemlidir. Daha önce böyle bir deneyim yaşanmamışsa bu bir risk olabilir.
- **Yeniden kullanılan yazılım miktarı:** Yeniden kullanım oranı iyi bir şeydir ve geliştirmeyi hızlandırabilir. Ama bu konuda bilgi sahibi olmadan kullanılırsa kritik bir hata nedeni olabilir.

Özetle program sayısı, satır sayısı, kullanıcı sayısı ve veri tabanı büyüklüğü, ürün büyüklüğünü etkileyen öğelerdir. Bunlar daha önce geliştirilen yazılımların ortalamasından fazlaysa risk oluşur.

10.2 İşletmecilik Riski

İşletmecilik riskleri şu sorularla belirlenir:

- Ürünün şirket geliri üzerindeki etkisi nedir?
- Dağıtım tarihi uygun mu?
- Bu ürünle birlikte çalışan diğer ürün ya da sistemlerin sayısı nedir?
- Son kullanıcıların düzeyi nedir?
- Dağıtım gecikirse etkilenecek maliyet nedir? Müşteriye geç teslim etmek nasıl bir yaptırım getirir?
- Müşteriye dağıtılacak ürünün dokümanlarının kalitesi nasıl istenmiştir? Ne miktarda belge istenmiştir?
- Ürünün pazarlamasını yapacak firmalar doğru seçilmiş mi?

Hoca şirket geliri sorusunu bir örnekle açtı: Yeni kurulan bir şirket için yeni bir ürün geliştiriliyor ve şirket bu ürün üzerinden değerlendirilecekse, bu durum işletmecilik açısından geliştiricilerde stres yaratabilir.

10.3 Müşteriyle İlişkili Risk

Tüm müşteriler eşit değildir. Gereksinimleri, kişilikleri ve satıcıyla ilişkileri açısından farklılık gösterebilirler. Farklı müşterilerle ilgili riskleri belirleyen bir risk kontrol listesi hazırlanmıştır:

- Bu müşteriyle daha önce çalışıldı mı?
- Müşteri neye gereksinim duyduğu hakkında somut fikir sahibi mi?
- Müşteri, proje ara kontrollerine ve gözden geçirmelere katılmaya istekli mi?
- Müşteri değerlendirici olarak seçilen kişi, ürün alanında teknik olarak bilgili mi?

Ara kontrol sorusu proje süresi açısından önemlidir. Kilometre taşına gelindiğinde o zamana kadar geliştirilenlerin değerlendirilmesi istenir. Müşteri bunu zamanında yapmazsa süreç boşuna gecikebilir. Değerlendirici seçilen kişi yazılım geliştirme sürecinden hiç haberdar değilse, sizden mümkün olmayan şeyler isteyebilir.

Bu sorulara "evet" cevabı veriliyorsa durum iyidir. Birçoğuna "hayır" cevabı veriliyorsa müşteriyle ilişkili risk vardır.

Hoca birinci soruya bir incelik ekledi: "Daha önce çalışıldı mı?" sorusu, müşteriyi tanıyor muyuz anlamındadır. Müşteriyle daha önce çalışılmış ama alttaki sorular açısından riskli bir müşteri olduğu biliniyorsa, bu cevabın olumlu bir etkisi olmaz.

10.4 Süreç Riski

Bu başlıktaki ana soru şudur: Projenin yazılım geliştirme süreci iyi tanımlanmış mı? Analiz, tasarım ve test rastgele mi yapılıyor, yoksa kurallara ve bir süreç modeline göre mi? Soru listesi şöyledir:

- Projede kullanılacak yazılım sürecinin yazılı tanımı ve alt başlıkları belirlendi mi? Bu tanım herkese belgelenecek biçimde mi?
- Çalışanlar belgeleme yapmaya istekli mi?
- Yazılım süreç tanımı başka projelerde kullanıldı mı?
- Yöneticilere ve teknik personele yazılım mühendisliği eğitimi verildi mi? Gerekli belgelemeye teşvik eden bir eğitim oluşturuldu mu?
- Yazılım mühendisliği standartları biliniyor mu?
- Gereksinim analizi, tasarım ve kodlamada düzenli teknik gözden geçirmeler yapılıyor mu? Sonuçları belgeleniyor mu?
- Konfigürasyon yönetimi gerçekleştiriliyor mu?

Hoca ikinci soruda slaydı düzeltti. Slaytta "çalışanlar bu dokümanı projede kullanmaya istekli mi" yazıyordu. Hocaya göre cümle hatalıdır. Sorulması gereken, belgeleme yapmaya istekli olup olmadıklarıdır.

Üçüncü soruyu bir örnekle açtı: Bir şirket önce şelale (waterfall) kullanıyordu, sonra XP denemeleri oldu, sonra Scrum'a geçilmesine karar verildi. Bu, Scrum'la geliştirilen ilk proje mi? Bir süreç modeli daha önce başka projelerde kullanılmamışsa, ilk kez burada kullanılıyorsa bu sorunun cevabı önem kazanır.

Standart sorusunda önceki bir örneğe döndü. Proje planının en az hangi yapıda olması gerektiğini derste bir örnekle vermişti. IEEE yazılım standartları çerçevesinde bunun daha kapsamlı bir standardı da vardır. Konfigürasyon yönetimi ise geliştirilen yazılımla ilişkili bütün değişikliklerin takip edilebildiği yapıdır.

10.5 Teknik Risk

Teknik riskler, yazılım geliştirme süreciyle ilişkili daha teknik bilgileri bir arada tutan risk tanımlarıdır. Sorular şunlardır:

- Müşteri ile geliştirici arasındaki iletişime yardımcı olan spesifikasyon teknikleri kullanılıyor mu?
- Kod için belgeleme ve açıklama var mı?
- Tasarımda belirli bir yöntem kullanılıyor mu? Örneğin nesneye yönelik UML yaklaşımları ya da klasik yaklaşımlar.
- Test durumu (test case) tasarımı için yöntem kullanılıyor mu?
- Analiz ve tasarım sürecinde yazılım araçları, yani CASE araçları kullanılıyor mu?
- Prototip geliştirmek için hızlı yazılım geliştirme araçları kullanılıyor mu?
- Tüm yazılım projeleri için kalite ölçütleri belirlendi mi?

CASE aracı kullanmak işi geliştirir, rutin işi hızlandırır ve daha hatasız yapılmasını sağlar. Ama önemli bir koşul vardır: Kullanacak kişiler o araç hakkında bilgi sahibi olmalıdır. Hoca bunun da kayda geçirilmesi gerektiğini söyledi.

10.6 Personel Riski

Personel riski için Software Engineering Institute'un oluşturduğu sorular kullanılır:

- İnsanlar yeteneklerine göre kullanılıyor mu?
- Projede yeterli çalışan var mı?
- Personel çalışma süresinin ne kadarını bu projeye ayırıyor? Aynı anda birkaç proje yürüdüğü için yarı zamanlı mı çalışıyor?
- Verilen işler çalışanların beklentilerini karşılıyor mu? İşi severek mi yapıyorlar, angarya olarak mı görüyorlar?
- Personel, kullanacağı araçlar ve yaklaşımlar konusunda eğitim aldı mı?

Bunların çoğuna "hayır" cevabı verilmişse personel riski vardır.

Hoca slaytta yazmayan bir soru daha ekledi: Çalışanlar, özellikle kritik görevlerdekiler, proje süresinde ve projenin en yoğun zamanında işten ayrılabilir mi? Bunun için anlaşma yapmak gibi bir önlem alınmış mı?

11. Risk Derecesi: Olasılık ve Etki

Sayılan risk tiplerinin hiçbiri tanımlandığı anda gerçekleşmiş değildir. Sonradan gerçekleşebilirler. Ayrıca bütün projelerde hepsi karşımıza çıkmaz. Bunlar olabilecek risklerdir, bizim projemizde bazıları çıkabilir. Bu yüzden önce riskin derecesini anlamaya çalışırız. Bunun için iki soru sorulur:

- Riskin olma olasılığı nedir?
- Riskin projeye etkisi nedir?

Olasılık ve etki yüksek, orta, düşük olarak kategorize edilebilir. Ya da belli sayı aralıkları yüksek, orta ve düşük olarak tanımlanabilir. Olasılık ve etki çarpıldığında riskin derecesi ortaya çıkar:

$$\text{risk derecesi} = \text{olasılık} \times \text{etki}$$

> **Ek bilgi (derste anlatılmadı):** Bu çarpım, risk yönetiminde risk maruziyeti (risk exposure) olarak da bilinir. Çarpımın sayısal yapılabilmesi için etkinin bir ölçeğe ya da maliyete dönüştürülmesi gerekir. Yüksek, orta, düşük gibi sözel değerlerde çarpım yerine, iki değerin birleşimini gösteren bir olasılık-etki matrisi kullanılır.

Hocanın örneği personelin işten ayrılmasıydı. Slaytta yazmıyordu, ama hoca az önce üzerinde durmuştu. Ayrılan kişi proje yöneticisi olduğu için etki yüksektir. Olasılık ise yapılan anlaşmaya bağlıdır. Çok kuvvetli olmayan bir anlaşma yapılmışsa olasılık %50 ya da %30 olabilir. Olasılık ve etki değerlerinin birleşimi risk derecesini verir.

Sonra bir risk tablosu yapılır. Her satıra bir risk yazılır. Yanına kategorisi, olma olasılığı ve gerçekleştiğinde etkisi eklenir. Bu değerler üzerinde düşünülmesi ve bir sonuca varılması beklenir.

Tablo bir kez yapılıp bırakılmaz, takip edilmelidir. Riskin olasılığı şu anda arttı mı, azaldı mı? Risk gerçekleştiğinde etkisinde bir değişiklik var mı?

12. Riski Azaltma Örnekleri

Riskleri ortadan kaldırmak için ne yapılır? Hoca slayttan birkaç başlık verdi:

- **Mazeret süresi:** Örneğin bir donanım kurulum siparişi için iki ay süre verilmişse, sürenin bitmesine bir ay kala satış sorumlusuna işin olup olmadığı sorulur.
- **Talep izleme:** Projenin bağlı olduğu teslim edilecek bazı şeyler varsa, sağlayıcının bunu takip etmesi.
- **İnsan iletişimine destek:** Bu konuda gereklilikler ortaya çıkabilir.

13. Risk Tablosu Örneği ve Sadeleştirme

Hoca ikinci aradan sonra risk tablosunun hazırlanışını bir örnekle gösterdi. Risk yönetiminde riskleri işleme ve tabloyu oluşturma adımları şunlardır:

1. Olası riskleri tüm ekip adlandırır.
2. Çeşitli kategoriler belirlenir.
3. Herkes uzman olduğu risk üzerinde neler olabileceğine, gerçekleşme olasılığına ve etkisine bakar.
4. Bunlarla bir tablo oluşturulur.
5. Tablo sadeleştirilir.

Tablonun sütunları risk numarası, riskin adı, türü, etkisi ve olasılığıdır. Örnekte türler proje riski, teknik risk ve işletmecilik riski olarak özetlenmişti. Hoca, tablodaki risklerin bazılarının gerçekçi, bazılarının ise örnek olarak yazıldığını söyledi.

Risk	Tür	Etki	Olasılık
Proje organizasyonunun doğru belirlenmemesi	Proje riski	Orta	Yüksek
Büyüklük ve maliyet tahmini	Proje riski	Büyük	Düşük
Sonuçların doğru modellenmesi (güvenilirlik)	Söylenmedi	Söylenmedi	Söylenmedi
CASE aracı kullanımı ve eğitimi	Teknik risk	Söylenmedi	Söylenmedi
Pazarlama (ilk ürününü satmaya çalışan yeni şirket)	İşletmecilik riski	Büyük	Yüksek

Hoca satırları şöyle yorumladı:

- **Proje riski:** Projenin yürütülmesi sırasında verilen hatalı kararlarla ilişkilidir. Örneğin proje ekip organizasyonu iyi kurulmamış olabilir. Etkisi orta, olasılığı yüksek olarak verilmiştir.
- **Tahmin riski:** Bu da projeyle ilişkilidir. Projenin büyüklüğü ve maliyeti ölçülür, tahmin edilir. Etkisi büyük, olasılığı düşük yazılmıştır, yani genellikle doğru tahmin edildiği varsayılmıştır. Hoca bunun her zaman doğru olmadığını ekledi.
- **Güvenilirlik:** Sonuçların doğru modellenmesi konusunda risk var mı sorusudur.
- **Araçlar:** İşi kolaylaştıran CASE (computer-aided software engineering) araçları kullanılıyor mu? Kullanılıyorsa eğitimi alındı mı? Bunun da olasılığı ve etkisi vardır.
- **Pazarlama riski:** Bilgisayar mühendisi olarak bir şirket kurmuş ve ilk ürününü satmaya çalışıyorsanız, olasılık yüksektir. Etkisi de büyüktür. Ürünü geliştirirsiniz ama satacak yer bulamazsınız.

13.1 Tablonun Sadeleştirilmesi

Bazı riskler olasılık ve etki açısından orta ile düşük arasındadır. Örneğin personelin işten ayrılması gibi bir durum yoksa ya da personelin çalışma performansıyla ilgili bir tedirginlik yoksa, bu satırlar orta-orta ya da küçük-düşük kalır.

Bunun için riskler, etki ve olasılık durumlarına göre bir diyagrama yerleştirilir. Küçük-düşük tarafta ya da ortada kalanları sürekli incelemek gerekmez. Bunlara daha aralıklı bakılır.

> **Hocanın vurgusu:** İlk başta çok büyük bir tablo çıkar ve bu tabloyu kısa aralıklarla izlemek zordur. Etkisi ve olasılığı büyük olanlara öncelik verilir. Bunlar bütün gözden geçirme noktalarında denetim altında tutulur.

Böylece tablodan bir öncelik grafiği çıkarılır ve tablo küçültülür. Küçültülmüş risk tablosundaki her risk, her kilometre taşında değerlendirilir ve buna göre önlem alınır.

14. Gantt Diyagramı

Gantt şemasını çizebilmek için önce WBS oluşturulmalıdır. Bu üç iş demektir: Projedeki aktiviteleri hiyerarşik olarak sıralamak, her birinin zaman bilgisini doldurmak ve aktiviteler arasındaki ilişkileri netleştirmek.

Gantt çizelgesi, projedeki her faaliyetin ne kadar süreceğini, başlangıç ve bitiş zamanlarını ve hangi kaynakları kullanacağını takip ettiğimiz diyagramdır. Proje planı sırasında belirlenir. Projenin yürütülmesi sırasında da gerçek durumun plana uygun olup olmadığını izlemek için kullanılır.

Gantt diyagramı 1910'larda önerilmiştir. Bugün bu iş akış planını oluşturmak, diyagramı çizmek, takip etmek ve projeyi yönetmek için pek çok yazılım vardır. Gantt ile birlikte çalışan PERT diyagramları için en çok kullanılan ürün Microsoft Project'tir (MS Project).

15. Örnek: Kitap Satışı Web Sitesi

Hoca kullanımı, bir kitap satışı web sitesi projesinin Gantt şeması üzerinde gösterdi. Önce analiz, sonra tasarım, sonra geliştirme ve test yapılıyor. Birinci düzeydeki aktiviteler bunlardır. Her birinin altında, o adımda yapılan işler alt başlık olarak yer alır.

```
Analiz
  Fizibilite çalışması
  Gereksinimlerin belirlenmesi (5 gün)
  Analizin modellenmesi
  Analiz bitimi toplantısı (kilometre taşı)
Tasarım
  Ön tasarım
  Ayrıntılı tasarım
  Tasarım bitimi toplantısı (kilometre taşı)
Geliştirme ve test
  ...
  Birim test
Proje bitişi
```

Hoca analiz alt başlıklarına küçük bir not düştü: Aslında gereksinimler belirlenirken modelleme çalışması da yapılır.

Şemadaki toplantılara dikkat çekti. Fizibilite çalışması bir faaliyettir, gereksinimlerin belirlenmesi 5 günlük bir faaliyettir. Analiz bitimi toplantısı ise değerlendirme için konmuştur. Bu, gözden geçirme toplantısı ya da başka çalışan birisi varsa onun toplantısı olarak sayılabilir. Tasarım bitimi toplantısı da öyledir. Kilometre taşları şemada eşkenar dörtgenle gösterilir.

Şemanın üstünde tarihler vardır. Bu uygulamada zaman ekseni haftalık verilmiştir. Şemaya göre analiz aşamasındaki fizibilite çalışması 21 Mart'tan 29 Mart'a kadar sürer. Bu, hocanın söylediği gibi bir hafta bir gündür:

$$29 - 21 = 8 \text{ gün} = 1 \text{ hafta} + 1 \text{ gün}$$

Gereksinimlerin belirlenmesi, fizibilite çalışmasıyla bir öncelik-sonralık ilişkisine bağlanmıştır: başlangıç-başlangıç (start-start, SS), bir gün sonra. Yani fizibilite başladıktan bir gün sonra gereksinimlerin belirlenmesi başlar. Ardından analiz bitimi toplantısı yapılır ve analiz aşaması tamamlanır.

15.1 Şemadan Düzey ve Model Okuma

Hoca şema üzerinden iki soru sordu: Bu örnekte kaçıncı düzeye kadar ayrıntı verilmiştir? Hangi yazılım geliştirme süreç modeli uygulanmıştır?

Soru: Bu Gantt şemasında kaçıncı düzeye kadar ayrıntı verilmiş? Bir öğrenci dördüncü seviyeye kadar verildiğini söyledi.

Cevap: Hayır. Birinci düzeyde analiz, tasarım, geliştirme ve test vardır. İkinci düzeyde bunların alt başlıkları vardır. Ayrıntılı tasarımın altında hangi aktivitelerin olduğu gösterilmemiştir, yani üçüncü seviye yoktur. Ayrıntı ikinci düzeyde kalmıştır.

Soru: İş sırası analizden tasarıma, tasarımdan geliştirme ve teste, oradan projenin bitişine gidiyor. Bu hangi modeli çağrıştırıyor? Spiral model olabilir mi? Bir öğrenci çevik (agile) olup olmadığını sordu.

Cevap: Spiral değildir. Çevik yaklaşım ise aktiviteleri daha bütünleşik gösterir. Bu bir şelale (waterfall) sürecidir. Çünkü analiz bitiyor, tasarım başlıyor. Tasarım bitiyor, geliştirme ve test başlıyor. Her adım, başlamak için önceki adımın bitmesini bekliyor.

> **Hocanın vurgusu:** Sizden "Waterfall'a göre ikinci düzeye kadar ayrıntıyla bir Gantt diyagramı oluşturun" dendiğinde, öncelik-sonralık ilişkileri bu biçimde kurulmalıdır.

15.2 Başlangıç Zamanı Nasıl Verilir?

Bir işin başlangıç zamanı her zaman tarih olarak verilmez. Başlangıç, öncelik-sonralık ilişkisiyle tanımlanır: Hangi faaliyet, hangi faaliyetten sonra, nasıl bir ilişkiyle gelecek? Örneğin yukarıdaki SS ilişkisi "biri başladıktan bir gün sonra başlayacak" demektir. Analiz ile tasarım arasında ise "analiz bittikten sonra tasarım başlayacak" tanımı yapılır.

Gantt diyagramıyla çalışmak için önce kullanılan süreç modeline uygun aktiviteler ve alt aktiviteler sıralanır. Genel olarak projeye bir başlangıç tarihi verilir. Sonrası aktiviteler arası ilişkilerden ilerler. Toplam süre de bu ilişkilerden hesaplanır.

16. WBS Kaç Düzey Olmalı?

Soru: Kaç seviye olacağına işin ayrıntısına ya da projenin büyüklüğüne göre mi karar veriyoruz? Burada iki seviye verilmiş. Beşinci seviye olup olmayacağını neye göre belirliyoruz?

Cevap: Bu, çalışma ekibinin projeyi nasıl takip etmesini istediğimize bağlıdır. Projeyi yakından ve daha sınırlı adımlarla takip etmek istiyorsak bütün alt başlıklara kadar inebiliriz. Ancak zorlayarak alt başlık üretmemek gerekir. Waterfall'da ikinci düzeye zaten rahatça gelinir. Üçüncü düzeye de gelinebilir. Alttaki aktiviteler 3 ya da 4 kademeye kadar görülebilir.

Hoca üçüncü düzey için örnekler verdi:

- **Fizibilite çalışması:** Teknik fizibilite ve ekonomik fizibilite ya da şöyle bir kırılım: alternatif çözüm yollarını belirleme, bu yollar için fizibilite analizi oluşturma, her birinin teknik ve ekonomik fizibilitesini gerçekleştirme.
- **Ön tasarım:** Yapılacak alt işlemlerin hepsi maddelenir. Birbirleriyle ilişkileri ve zaman aralıkları tanımlanırsa üçüncü düzeye inilmiş olur.
- **Ayrıntılı tasarım:** Veri tabanı tasarımı, ekran tasarımı, program tasarımı gibi alt başlıklara bölünebilir. Bir madde de tasarım raporu oluşturma olabilir.

  > **Hocanın vurgusu:** Çok büyük olmayan projelerde çok alt adımlara kadar inilirse, bu da planın kullanımı sırasında sorun olabilir.

17. Faaliyetler Arası İlişkiler ve Kritik Yol

Hoca aynı şema üzerinden faaliyetler arası ilişkileri gösterdi. Dört ilişki türü vardır: bitiş-başlangıç (finish-start), başlangıç-başlangıç (start-start), bitiş-bitiş (finish-finish) ve başlangıç-bitiş (start-finish).

Başlangıç-başlangıç örneği: Gereksinimlerin belirlenmesi ve fizibilite çalışması aynı zaman sürecinde devam edebilir. Ama ilişkiye bir ekleme yapılmıştır: Önce 2 numaralı adım olan fizibilite çalışması başlar, bir gün sonra gereksinimlerin belirlenmesi başlar. Adım numaraları şemadaki satırlardan sayılır.

Bitiş-başlangıç örneği: Hoca ilişki adını burada kullanmadı. Ama "analiz bittikten sonra tasarım başlayacak" tanımı bu türe karşılık gelir.

Bitiş-bitiş örneği: Şemada birim testin yanında bir bitiş-bitiş ilişkisi vardır. Hoca bunu şöyle okudu: Birim testin olması için 11 numaralı aktivitenin bitmiş olması gerekir. Yani yazılım geliştirme işi tamamen bitince birim teste başlanır. Ayrıca 10. iş de başlamış olmalıdır.

> **Ders sonrası not:** Hoca bu ilişkiyi finish-finish diye gösterip "geliştirme bitince birim teste başlanır" diye okudu. Bu okuma finish-start ilişkisine karşılık gelir. Finish-finish ise bir işin bitişini diğerinin bitişine bağlar. Şemadaki ilişki slaytta kaldığı için kesin değil. Büyük olasılıkla slaytta birim testin bitişi geliştirmenin bitişine bağlanmıştı.

> **Ek bilgi (derste anlatılmadı):** Standart tanımlarıyla B işi A işine şöyle bağlanır. Finish-start: A bitmeden B başlayamaz. Start-start: A başlamadan B başlayamaz. Finish-finish: A bitmeden B bitemez. Start-finish: A başlamadan B bitemez. Bu son tür en az kullanılanıdır; derste de yalnız adı geçti.

17.1 Kritik Yol

Bir de kritik yol vardır. Projenin tamamlanması için yol üzerindeki faaliyetlerin ne kadar süreceğiyle ilgilidir. Hocaya göre bunu Gantt diyagramında tam çizebilmek için, aktivitelerin sıralaması ve dengelemesi konusunda PERT diyagramıyla bir çalışma yapılmış olması gerekir.

> **Ek bilgi (derste anlatılmadı):** Kritik yol, ağdaki başlangıçtan bitişe en uzun süren faaliyet zinciridir ve projenin en kısa bitiş süresini belirler. Bu zincirdeki bir faaliyet gecikirse proje de gecikir. PERT, bu tür hesapları faaliyetlerin öncelik ağı üzerinde yapar. Gantt diyagramı ise adını Henry Gantt'tan alır.

18. Proje Planlama Araçları

Bu diyagramlar elle, kareli kâğıtlara çizilerek ve zaman dilimleri oluşturularak da yapılabilir. Ama günümüzde böyle yapmak pek uygun değildir, araç kullanılır.

MS Project, proje yönetiminde kullanılan en yaygın araçlardan biridir. Hoca bunu yaygın olduğu için söylediğini belirtti. Başka birçok proje yönetimi ve proje planlama aracı vardır ve proje planı bunlardan herhangi biriyle yapılabilir. Derste MS Project'in tercih edilme nedeni, laboratuvarda Microsoft ürünleriyle birlikte yüklü olması ve iş yerlerinde çok yaygın kullanılmasıdır.

> **Hocanın vurgusu:** MS Project bir CASE aracı değildir, bir yazılım geliştirme aracı da değildir. Bütün projelerde zaman ve kaynak planını doğru göstermeye yarayan bir proje yönetimi aracıdır. Mimari, inşaat ya da kimya projesi fark etmez.

Hoca alternatif araçları sordu. Altyazıda başka bir ürün adı geçmiyor.

Dersin Toparlaması

Bu derste proje planının parçaları bir kez daha izlendi: Sorun müşteriyle tanımlanır, kısıtlar, varsayımlar ve kabul kriterleri kayda geçirilir. Çözüm yolu alternatifler karşılaştırılarak gerekçesiyle seçilir. Geliştirme süreci planlanır. Sonra risk analizi yapılır. Riskler kategorilere ayrılır, olasılık ve etkiyle derecelendirilir. Büyük tablo sadeleştirilir ve kalan riskler her kilometre taşında denetlenir.

Dersin son bölümü planın zaman boyutuna geçti. WBS, seçilen süreç modeline uygun bir hiyerarşi kurar. Gantt diyagramı bu hiyerarşiye süre, kaynak ve faaliyetler arası ilişkileri ekler. Başlangıçlar çoğu zaman tarihle değil ilişkiyle verilir, toplam süre de buradan hesaplanır. Hoca sonraki derste zaman planlaması, WBS ve kullanılacak iş planlama araçları üzerinde duracağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.